FIT2099 Bootcamp week 5 requirement 3c Design Documentation

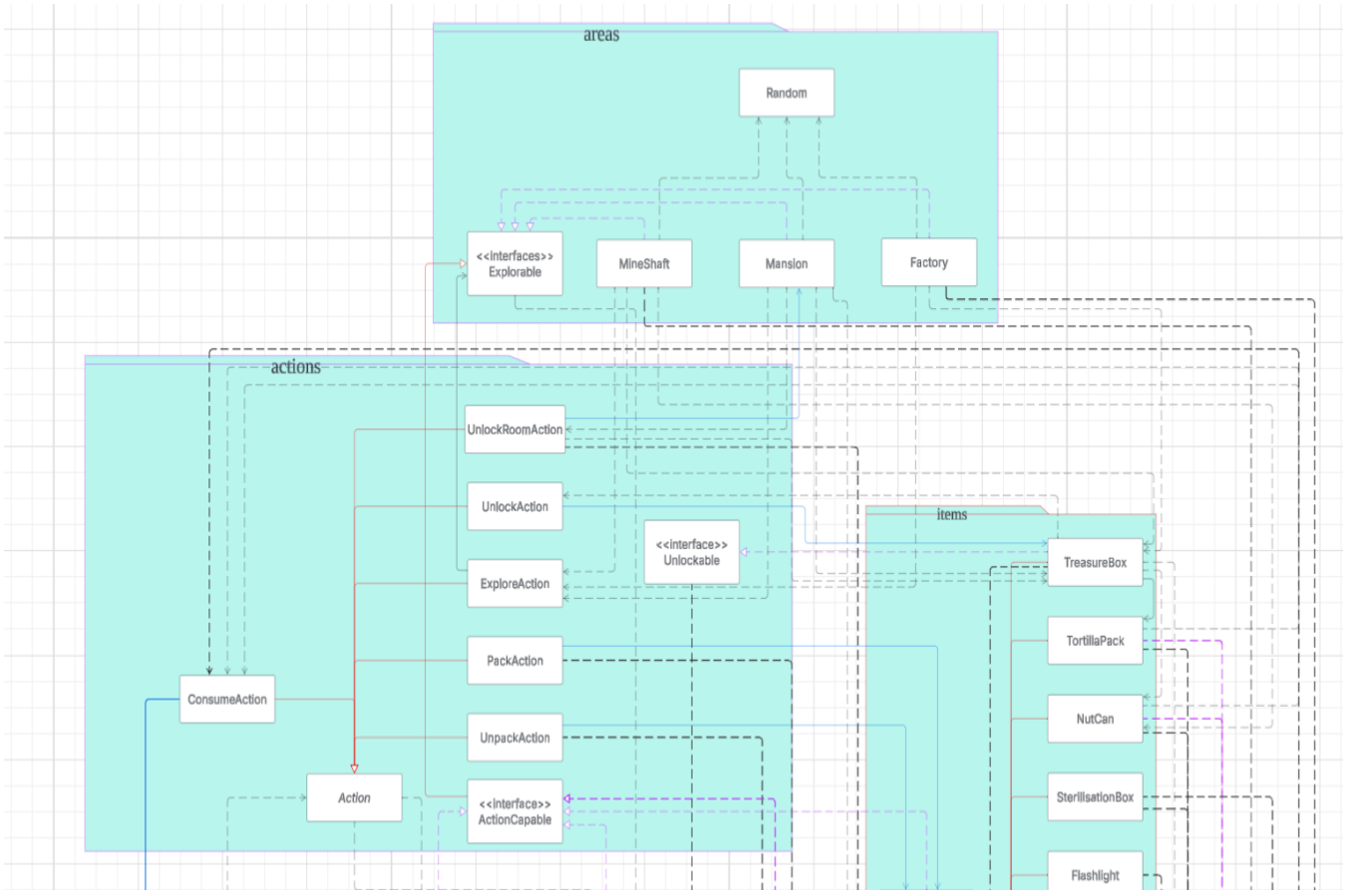
Project Summary

This project simulates a crew member (Steve) exploring the moon surface, collecting items in a weight-limited backpack, and managing energy through consumables. This simulation runs a turn-based loop where the user selects actions from a text-generated menu and ends when the crew member runs out of energy.

Design Goals:

- Extensibility -> new unlockable items or areas must be added without modification of existing classes
- Reusability -> the same unlock mechanism should work for both TreasureBox and any other unlockable item in the future without code duplication
- Informative UI -> the menu should clearly show whether the lockpick is already broken or still usable

Design Diagram



Design Rationale

A. Implementation of unlocking TreasureBox

Design 1: Create an Unlockable interface and a concrete UnlockAction class. TreasureBox implements Unlockable. UnlockAction depends only on the interface.

Pros	Cons
Interface Unlockable provides a clear blueprint of method that need to be implemented to unlock unlockable items	Requires creating more files
The same UnlockAction can unlock any item that implements Unlockable interface	
Changing unlock mechanism only requires modifying UnlockAction, not every unlockable item	

Design 2: Directly embed the unlock logic inside TreasureBox.allowableActions()

Pros	Cons
Fewer files, since only the unlockable item class need to exist	Each unlockable item would need to implement its own unlock logic
easy implementation and simple logic	Same unlock logic copied through every unlockable item
	Tighter coupling since actions is made inside TreasureBox, changes is riskier

B. Implementation of Mansion Room Unlocking

Design 1: Create a dedicated UnlockRoomAction class

Pros	Cons
Promotes reusability, easier to implement unlock room feature for future areas	Requires additional files
Changing from 3 TreasureBox to 5 only requires modifying the UnlockRoomAction class, not every involved class	Might be overkill if only one area ever needs room unlocking
Mansion only handles exploration while UnlockRoomAction focuses on unlock mechanic	

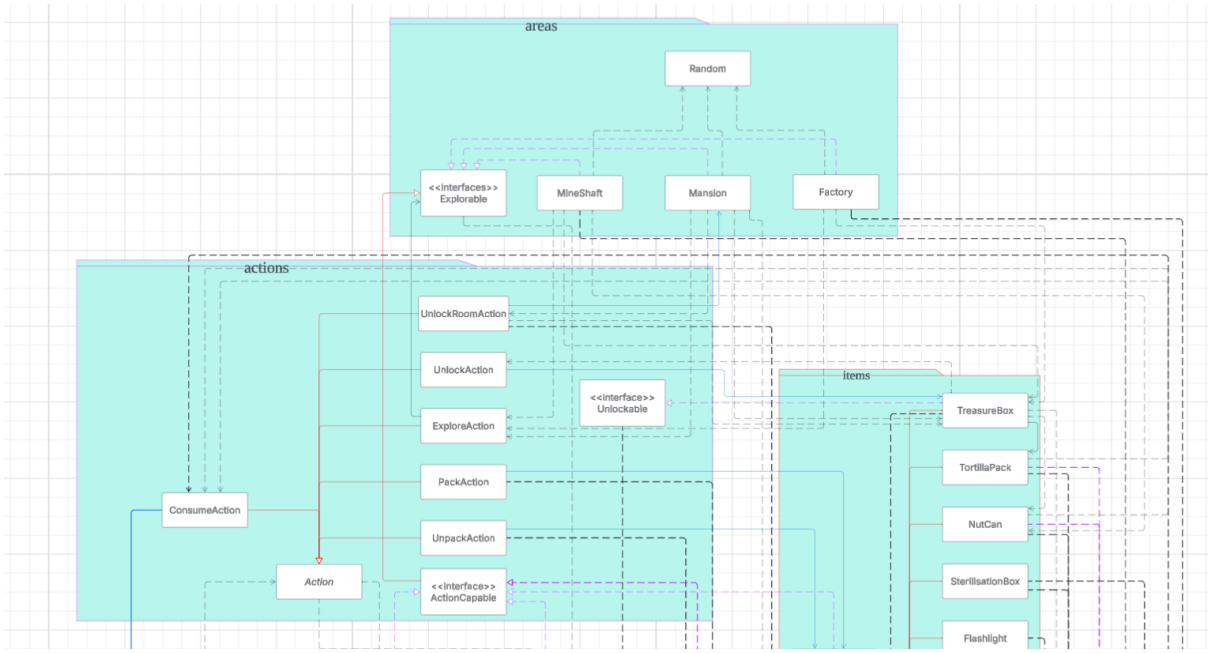
Design 2: Add unlock logic and action inside Mansion.allowableActions()

Pros	Cons
Fewer files, since the logic is directly embedded inside the class	Discourage reusability, if other areas want to implement this feature, the code must be copied
	Mansion now handles both exploration and room unlocking logic
	Messy allowableActions () method, since the unlock logic would be added inside instead of a Unlock Object
	If a new unlockable room type is added, existing classes must be modified

C. One-Time Use of Lockpick

Keep the Lockpick object but remove its UNLOCK capability using removeCapability (). Override Lockpick.toString() to display "usable" or "broken".

Pros	Cons
Provides clear UI feedback on whether the lockpick is broken or usable	Loops through the backpack to find items with the UNLOCK capability
Reuses existing method such as hasCapability () and removeCapability ()	
Searches for tools with UNLOCK capability, so future items with same capabilities will work as well	



The Diagram above shows the final design of Requirement 3c, which utilizes both Design 1 of parts A, B and Design of part C states above to implement an Unlockable interface, a concrete UnlockAction class for unlocking unlockable items, and a concrete UnlockRoomAction class for unlocking rooms in certain areas.

The new UnlockAction class extends the abstract Action class, and UnlockRoomAction also extends Action. TreasureBox implements the new Unlockable interface while also extending the abstract Item class. All these classes inherit their methods and attributes from existing abstract classes provided in the engine. We abstract these identities to avoid repetitions in code (DRY), as these classes share common behaviour with the respective abstract classes they extend.

For TreasureBox unlocking, the Unlockable interface provides a contract that any unlockable item must fulfil. The UnlockAction class depends only on this interface instead of the concrete class TreasureBox. While it requires more files, the benefits outweigh this minor cost. This design follows the Dependency Inversion Principle (DIP) because UnlockAction depends on the Unlockable abstraction rather than concrete TreasureBox details. It also adheres to the Open/Closed Principle (OCP) since new unlockable items can implement Unlockable without modifying UnlockAction. The Single Responsibility Principle (SRP) is also respected as TreasureBox manages its contents while UnlockAction handles execution logic.

For Mansion room unlocking, this design follows SRP by separating exploration concerns (Mansion) from unlock execution (UnlockRoomAction). It also supports OCP since adding room unlocking to other areas (e.g., Laboratory, Cave) requires creating a new action class, not modifying existing code. The pattern established here can also be reused across any area that needs room unlocking.

For one-time use of Lockpick, the design keeps the Lockpick object but removes its UNLOCK capability after use. The overridden toString() method displays "(broken)" instead of "(usable)", providing clear UI feedback that fulfils the "Informative UI" design goal. This approach reuses the existing capability framework (DRY) and ensures that broken lockpicks remain as scrap metal that still occupies backpack weight.

Limitations and Trade-offs

The current design assumes that only one lockpick is consumed per unlock action (the first found in the backpack). If the player has multiple lockpicks, they cannot choose which one to use. This is acceptable for the current scope but could be enhanced with a sub-menu in future iterations.